Introduction to Computer Science and Programming

Lab 11 – C++ Functions

Laboratory 11, February 26, 2026

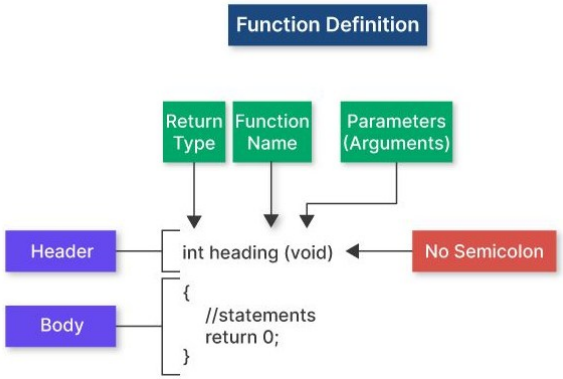
Filippo Ricca & Muhammad Mohsin

DIBRIS, University of Genova

Today…

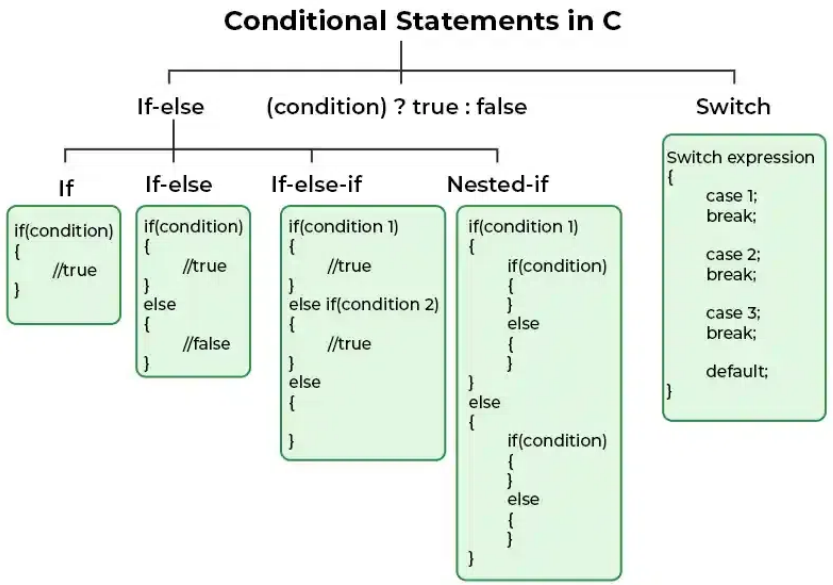
Today's lab is dedicated to **functions exercises in C++.**

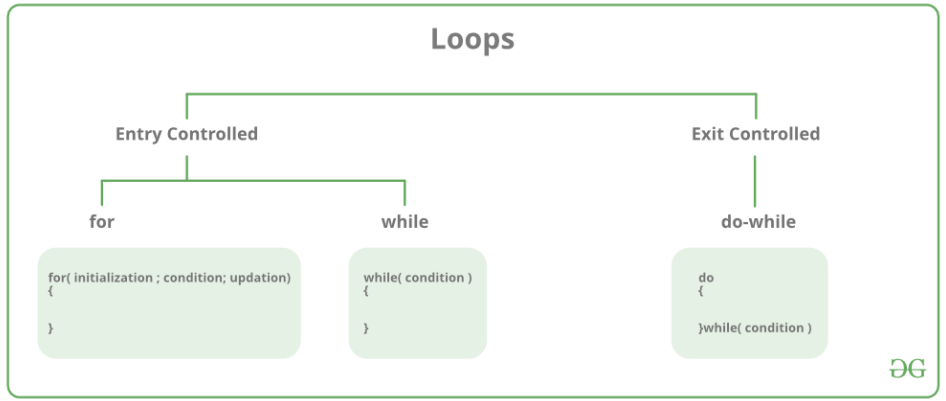
The exercises are divided into two categories: **easy** ones for students who have never programmed before and **more complex** ones for students who already have programming experience. The recommendation is to read both and then start with the ones that seem most appropriate.



C++ language (commands covered so far)- Lecture Recap

- Variables (local and global)
- Basic datatypes
  - Int, float, double, char, …
- Blocks
  - { … }
- Control flow commands
  - If, If-else, nested If, Ternary operator, Switch
- Loops
  - For, While, Do-While
- cout and cin commands
- Strings
- **Today: Functions**





Function

- Functions can be used to **define reusable code** and organize and **simplify code**
- Concept already seen in the first lessons at level of **Algorithm** and **EduBlocks**
- ****Problem**:** Given three integers, calculate the difference between the **largest** and the **smallest**

Complete Algorithm

Final Version:

1. **GET** A, B, C;
2. BIG **<-** COMPUTE_MAX(A, B, C);
3. SMALL **<-** COMPUTE_MIN(A, B, C);
4. RESULT **<-** BIG – SMALL;
5. **DISPLAY** RESULT

COMPUTE_MAX(A, B, C):

1. **IF** A>B **THEN** BIG <- A
   ELSE BIG <-B;
2. **IF** C>BIG **THEN** BIG <- C;
3. **RETURN** BIG

COMPUTE_MIN(A, B, C):

1. **IF** A<B **THEN** SMALL <- A
   ELSE SMALL <-B;
2. **IF** C<SMALL **THEN** SMALL <- C;
3. **RETURN** SMALL

It is a portion of code to which we can give a name and call (invoke) it

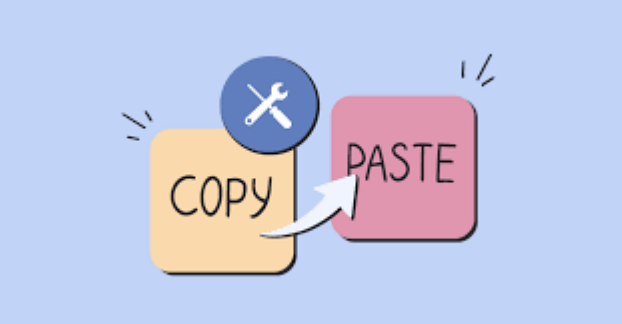
Find the sum of integers from **1** to **10**, from **20** to **37**, and from **35** to **49**, respectively

Example

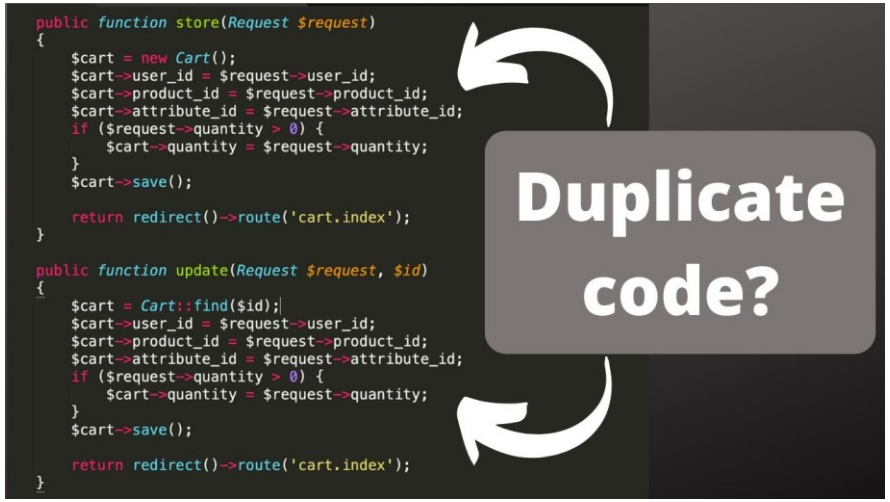
Write 3 loops

```
int sum = 0;
for (int i = 1; i <= 10; i++)
  sum += i;
cout << "Sum from 1 to 10 is " << sum << endl;

sum = 0;
for (int i = 20; i <= 37; i++)
  sum += i;
cout << "Sum from 20 to 37 is " << sum << endl;

sum = 0;
for (int i = 35; i <= 49; i++)
  sum += i;
cout << "Sum from 35 to 49 is " << sum << endl;
```

Duplicated code

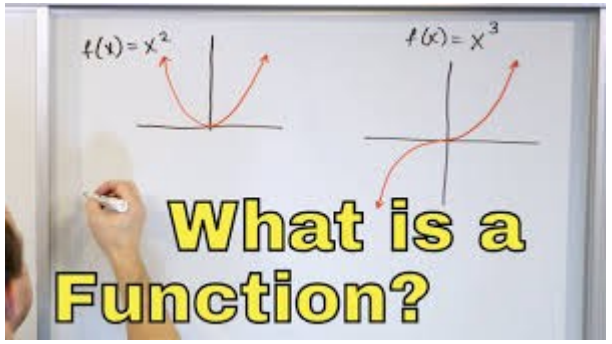


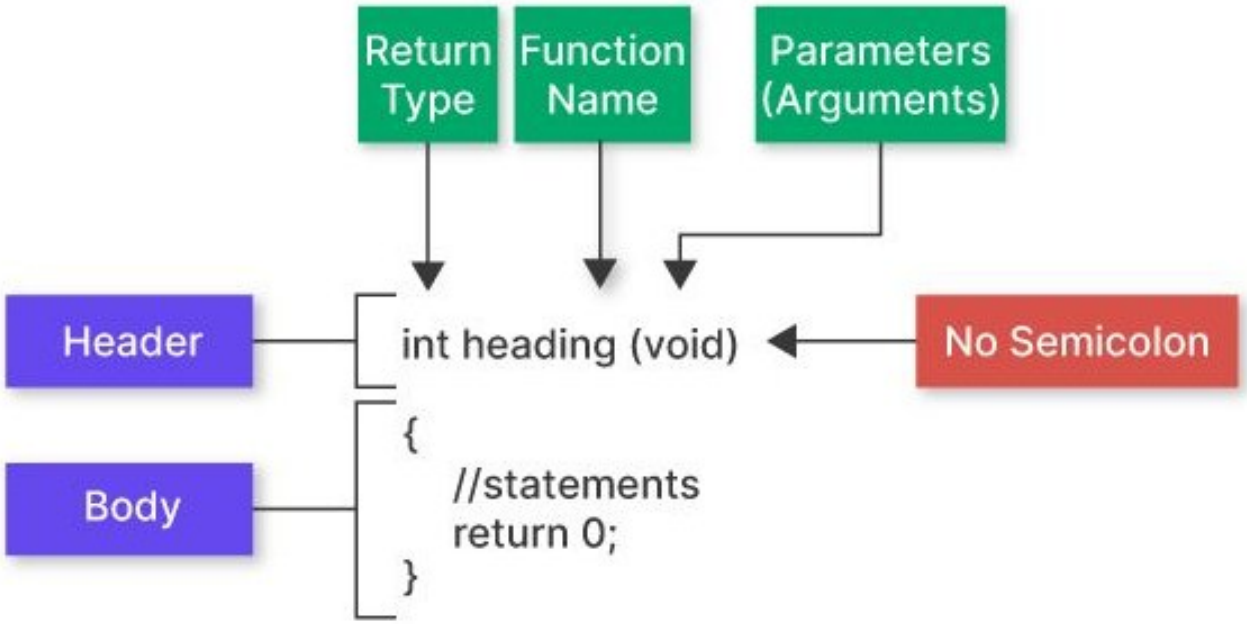
- Is it a good solution? **NO!**
- A lot of duplicated code!!
- Duplicated code is never a good solution because:
  - It makes the program longer than necessary and
  - **It increases the risk of propagating errors in the code**



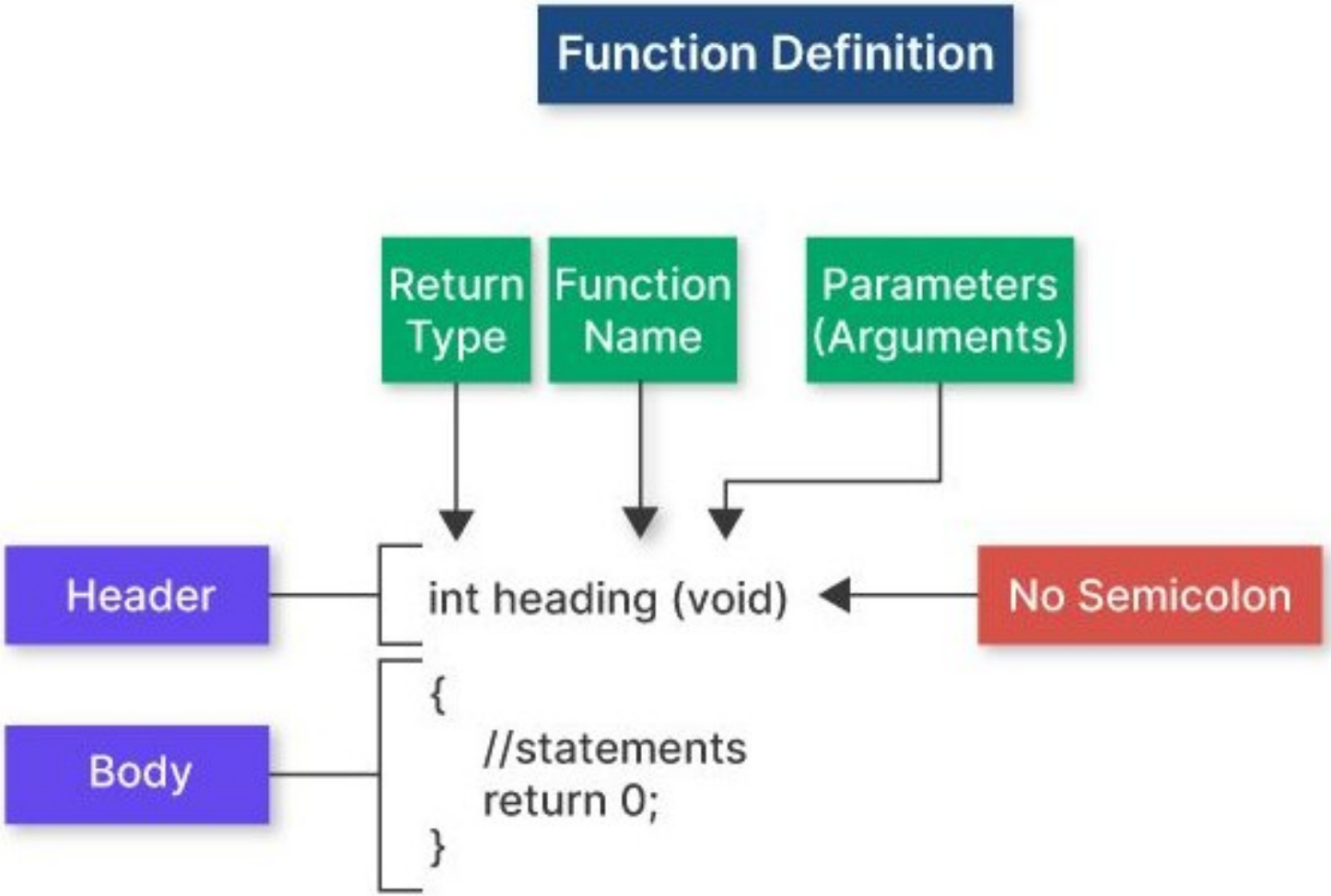
Defining a function

- A function is a **collection of statements** that are grouped together to **perform an operation** with **a name**
- A function definition consists of its **function name**, **parameters**, **return type**, and **body**





Function and Parameter Declaration



Function and Parameter Declaration

Function Declaration (Function Prototype)

A function declaration tells the compiler about the function's name, its return type, and the parameters it accepts. However, it does not include the actual implementation (body) of the function.

For Example : int sum(int a, int b);

Function declaration should always be before the main function

Function and Parameter Declaration

Function Definition

A function definition includes both the function header (same as the declaration) and the body, which contains the actual implementation of the function.

For Example :

```
int sum(int a, int b) {
                    int sum;
                    sum=a+b

                    return sum;
}
```

In function definition, should have function name, function parameters to be passed and a return type

Function and Parameter Declaration

Function Call

- Declared functions are not executed immediately. They are **"saved for later use",** and will be executed later, when they are called.

- To call a function, write the **function's name** followed by two parentheses **()** and a **semicolon ;**

Example

```cpp
#include <iostream>
using namespace std;

// Function declaration
int sum(int a, int b);

// Main function
int main() {
    int a = 10;
    int b = 20;

    // Function call
    int result = sum(a, b);

    cout << "Sum = " << result << endl;

    return 0;
}

// Function definition
int sum(int a, int b) {
    return a + b;
}
```

```cpp
#include <iostream>
using namespace std;

// Function definition
int sum(int a, int b) {
    return a + b;
}
// Main function
int main() {
    int a = 10;
    int b = 20;

    // Function call
    int result = sum(a, b);

    cout << "Sum = " << result << endl;

    return 0;
}
```

Be careful, because with some compilers a function declared after its use might still work, but it is not standard

Exercise 1 (simple): Calculate the area of: rectangle, triangle and cylinder using function overloading

Write a C++ program that defines a function to calculate **the area of rectangle, triangle and cylinder** using function overloading technique

Steps:

1. Define a function prototype
2. Create three function for each part to calculate the area using function overloading technique.
3. Call the function in **main()**.

Example:

Input:

- Length = 7
- Width = 10
- Base = 12
- Height = 7, Radius = 3

Output:

Area of Rectangle: 70

Area of Triangle: 120

Area of Cylinder: 188.4

Exercise 2 (simple): Check if a number is prime

Write a C++ program that defines a function called **"isPrime"** that takes an integer as a parameter and returns **true** if it's a prime number, otherwise **false**

Steps:

1. Define a function prototype for **"isPrime"**
2. Use loop to check divisibility from 2 to n-1
3. If the number has any divisors other than 1 and itself, return **false**

Hint: In **main(),** ask the user for a number, call the function, and display whether it's prime or not

Example:

Input:

- n = 14

Output: false

Exercise 3 (simple): Count the number of digits in an integer

Write C++ program that defines a function called **"totalDigits"** that takes an integer as input and returns the total number of digits in it.

Function should handle the zero input and negative, for example -987.

Example:

Number -987 has 3 digits

Number 0 has 1 digits

Number 7 has 1 digits

Exercise 4 (simple): Calculator

Create a C++ calculator that takes a number, a basic math operator (+,-,*,/,^), and a second number all from user input, and have it print the result of the mathematical operation. **The mathematical operations should be wrapped inside of functions**

It is important to handle error cases when a symbol that does not represent an operation (e.g., &) is entered, terminating the program using the exit() command as discussed in class

Example:

Input:

- n1 = 2
- Op = '+'
- n2 = 3

Output: 5

Exercise 5 (Simple): Subsequence

Write a C++ program that defines a function to check if whether a given string is a subsequence of another given string.

Return 1 for true and 0 for false.

Example:

Input:

word3: ACGGTGTCGTGCTATGCTGATGCTGACTTATATGCTA
subse3: CGTTCGGCTATGCTTCTACTTATTCTA

Output:

Is subse3 is the subsequence of word3? 1

Exercise 6 (More Complex): Add two binary numbers

Write a C++ program that defines a function that takes **two binary numbers as input, adds them together**, and return the result

Hint: represent binary numbers with strings (data type)

Example:

Input:

Enter the first binary number: 000011010

Enter the second binary number: 011100001

Output:

Sum of the binary numbers: 011111011

Exercise 7 (More Complex): Palindrome

Write a C++ function to check if a given string is a **Palindrome** or not.

A palindrome is a word, number, phrase, or other sequence of characters which reads the same backward as forward, such as madam, racecar

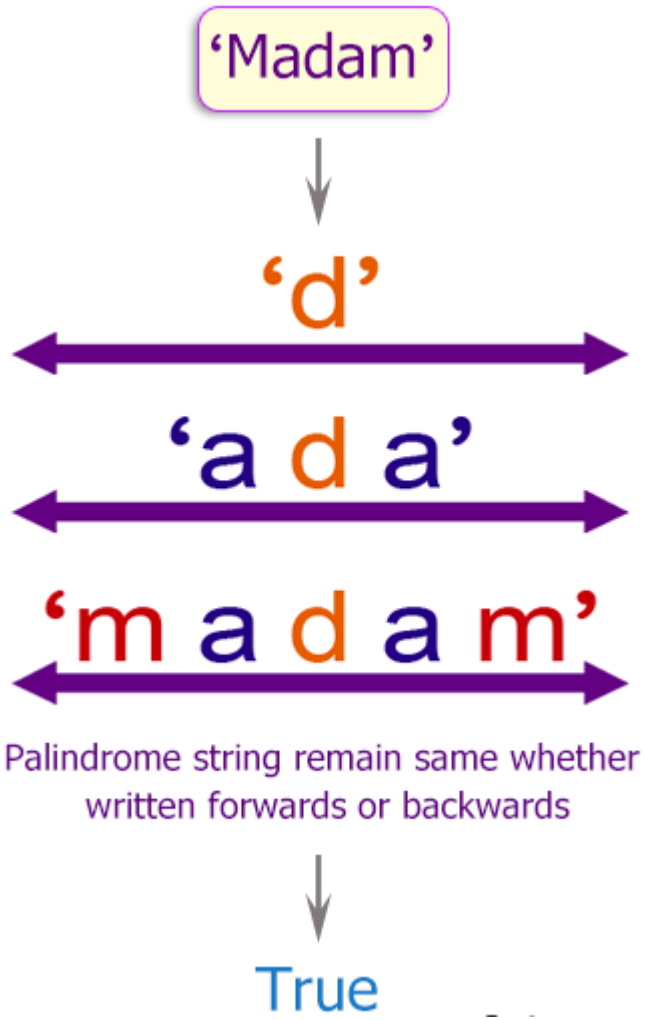
Example:

Input:

madam

Output:

True



Exercise 8 (More Complex): Roman numeral conversion

Write a C++ function to convert a given integer to a Roman numeral

From Wikipedia:

Roman numerals are a numeral system that originated in ancient Rome and remained the usual way of writing numbers throughout Europe well into the Late Middle Ages. Numbers in this system are represented by combinations of letters from the Latin alphabet.

Symbol	I	V	X	L	C	D	M
Value	1	5	10	50	100	500	1000

Example:

Input:

n: 7

Output:

VII